

תכנות מערכות מתקדם - תרגיל מספר 2

בתרגיל זה שני חלקים

חלק א

בחלק זה עליכם להמשיך להשתמש בפרוייקט ה JIRA מתרגיל 1, ולתכנן בו את חלק ב של תרגיל זה. תהליך הפיתוח של חלק ב יהיה בצמוד לפרוייקט ה JIRA, ואתם חייבים להשתמש ב JIRA לאורך כל הדרך בזמן אמת. הדבר יבדק על ידי הבודק.

אם ברצונכם להמשיך להשתמש ב Github מתרגיל 1 אתם יכולים לעשות זאת. במקרה זה עליכם ליצור branch נפרד עבור תרגיל 1 ולא לגעת בו (ב branch הנפרד) - כדי שהבודק יוכל לבדוק אותו. ואז אתם ממשיכים לעבוד על ה main branch בשביל תרגיל 2.

לחילופין, אתם יכולים לא לגעת ב main branch וליצור branch חדש שיהיה ה main branch עבור תרגיל 2.

בכל מקרה, שימו לב לא לגעת בbranch שמיועד לתרגיל 1 לאחר מכן - כדי לא לפגוע בימי ה"חוסד".

אם אתם רוצים, אתם יכולים ליצור repository נפרד עבור תרגיל 2.

אם החלפתם קבוצה ואין באפשרותיכם להשתמש בעקבות כך בפרוייקט ה JIRA (או Github וכו') מהתרגיל הראשון, צרו פרוייקטים חדשים בהתאם עם הקוד מהתרגיל הקודם.

בפרט, עליכם להקפיד על:

- חלוקה ל epics
- כתיבת users stories
- כתיבת tasks
- יצירת ספרינט, מינוי scrum master מבין חברי הקבוצה, תכנון הספרינט, קיום מפגשי סטטוס לאורך הספרינט (לא כל יום, אבל לפחות פעמיים בשבוע) חובה לתעד ב JIRA את כל הפעולות הללו ואת התוצאות שלהן. למשל, התובנות והמסקנות של מפגשי הסטטוס חייבים להיות מפורטים ומתועדים.
- עדכון סטטוס משימות (in progress, code review, done) כולל עדכון של activity של משימות תוך כדי ביצוע המשימה. שימו לב, עליכם להוסיף את הסטטוס code review, הוא לא מגיע ברירת מחדל. משימה עוברת ל code-review כאשר סיימתם אותה ופתחתם לה pull request. באפשרותכם כעת לשייך את המשימה ב JIRA אל חבר צוות אחר שצריך לעשות review לקוד שלכם. לחילופין, תוכלו להשאיר את המשימה אצלכם ול"תייג" את החבר צוות האחר ב activity של הissue ב JIRA. בכל מקרה, עליכם להגדיר בצורה ברורה מי לבסוף מעביר את המשימה ל done.
- שיוך משימות לסטודנט שמבצע אותן לפני שהוא מתחיל לעבוד עליהן.
- עליכם לזהות אילו משימות "תוקעות" משימות אחרות (is blocked by) ולהביא את זה לידי ביטוי בפיצ'ר הרלוונטי ב JIRA.
- שיוך כל המשימות ל feature branches. כלומר, יש ל Github ול JIRA התממשקות כך שאם רושמים את שם הissue ב JIRA בשם של ה branch והמשימות, JIRA יודע להציג את זה ב JIRA issue אוטו.

חלק ב

בחלק זה אתם מתבקשים להפוך את מערכת ההמלצות מתרגיל 1 לקליינט-סרבר על גבי TCP, כאשר הסרבר הוא multi-threaded. השרת יטפל בכל הלוגיקה העסקית, כלומר ינהל את המידע ויבצע חישובים. השרת יהיה ממומש ב C++.

הלקוח יאפשר למשתמש להזין פקודות ולהציג את הפלט שלהן (הפלט שהשרת שלח). הלקוח ימומש ב Python 3.

השרת יאזין לבקשות מהלקוח על גבי סוקט tcp. הפורט שהשרת מאזין לו יועבר כפרמטר לתוכנית (ארגומנט לפונק' ה main).

הלקוח יקבל כארגומנטים לחיבור את כתובת ה ip של השרת ולאחר מכן את מספר הפורט שהשרת מאזין לו.

כאשר הלקוח מופעל, הוא מקים חיבור TCP מול השרת וממתין לקבל פקודה מהמשתמש דרך ה console. כאשר המשתמש מזין פקודה, הלקוח שולח אותה לשרת, השרת מבצע אותה ומחזיר את התשובה ללקוח שמדפיס אותה למסך.

השרת יטפל בכל לקוח בעזרת thread נפרד, כך שניתן להריץ בו זמנית כמה לקוחות שנרצה, וכל הלקוחות יכולים לתקשר מול השרת **בו זמנית**. ודאו שאתם מטפלים בגישה למשאבים משותפים בהתאם.

שימו לב: הלקוח מקים בהתחלה חיבור tcp מול השרת והלקוח משתמש **באותו** חיבור מול השרת לאורך כל הריצה של הלקוח. כלומר, כל הפקודות נשלחות מהלקוח אל השרת **על גבי אותו חיבור**.

הלקוח לא מקים חיבורים נפרדים לפקודות.

הלקוחות **אינם** multi-threaded. כלומר, הלקוח מקבל פקודה אחת מהמשתמש, שולח אותה לשרת וממתין לתשובה מהשרת. עד שהתשובה הזו מגיעה ומוצגת למסך, הלקוח אינו קורא את הפקודה הבאה מה console.

שימו לב: הלקוחות בתרגיל זה הינם "טיפשים", כלומר, הם אינם יודעים מהי פקודה חוקית והאם יש פקודות מסויימות או אין. הם פשוט שולחים את מה שהמשתמש ביקש לשרת ומדפיסים את התוצאה כשיקבלו אותה.

השרת תומך **בכל הלוגיקה** שהוגדרה בתרגיל 1, עם השינויים הבאים:

- הקלט/פלט הוא מ/אל הסוקטים **ולא** מ/אל ה console. כל פקודה שנשלחת מהלקוח אל השרת מסתיימת בתו ירידת שורה, וכל פלט שהשרת שולח אל הלקוח מסתיים בתו ירידת שורה.
- הפקודה add תקרא POST **במקום** add. עם זאת, הפקודה תיחשב חוקית רק אם **לא** היה משתמש כזה עד כה. במידה והתקבלה פקודה חוקית היא עושה אותו דבר כמו בתרגיל 1, והפלט אל הלקוח הוא (בדיוק ככה):

201 Created

- פקודה חדשה בשם PATCH. הפקודה עושה אותו דבר כמו פקודת add מתרגיל 1, אבל הפקודה תיחשב חוקית רק אם **קיים כבר** משתמש כזה (כלומר המשתמש שהוזן לפקודה כבר קיים בגלל פקודת post קודמת). במידה והתקבלה פקודה חוקית הפלט אל הלקוח הוא (בדיוק ככה):

204 No Content

- פקודה חדשה שנקראת DELETE:

DELETE [userid] [movieid1] [movieid2] ...

ותמחק את הצפייה של משתמש [userid] בסרט [movieid1] ובסרטים הנוספים במידה ומופיעים בפקודה. במידה והתקבלה פקודה חוקית, הפלט של השרת אל הלקוח הוא (בדיוק ככה):

204 No Content

- הפקודה recommend תקרא GET **במקום** recommend. במידה והתקבלה פקודה חוקית, הפלט של השרת אל הלקוח יהיה (בדיוק ככה):

200 Ok

- ואז פעמיים התו \ (ירידת שורה) ואז אותו פלט מהתרגיל הקודם.
- הפקודה help נקראת אותו דבר ומחזירה את אותו הפלט (כמעט) אל הלקוח, אבל כוללת את הפקודות החדשות, וכל הפקודות בפלט מופיעות בסדר אלפביתי, למעט help שמופיעה בסוף. השינוי היחיד למבנה הפלט מהתרגיל הקודם הוא שכל פקודה שיש לה פרמטרים (למשל ל GET יש פרמטרים אבל לקוד help אין), תתווסף המילה arguments באופן הבא (בדיוק ככה לכל הפקודות עם פרמטרים):

GET, arguments: [userid] [movieid]

- במקרים בהם מבנה הפקודה תקני אבל לוגית הוא לא הגיוני כי זה לא מסתדר עם המידע הקיים על ה userid, למשל אם רוצים לעשות PATCH למשתמש שלא עשו לו POST, או רוצים לעשות DELETE למשהו שלא עשו לו POST/PATCH אזי יש להחזיר ללקוח (בדיוק ככה):

404 Not Found

- בכל מקרה שתזון פקודה אחרת או פקודה שאינה תקנית על השרת להתעלם ולשלוח ללקוח (בדיוק ככה):

400 Bad Request

חובה להקפיד על הקלט הנ"ל במדויק ועל הפלט הנ"ל במדויק. אין לסטות ממנו ו/או להדפיס שום דבר נוסף/אחר/שונה, גם לא הנחיות, שגיאות או כל דבר אחר. התוכניות לא עוצרות לעולם.

הנחיות:

- יש להמשיך ולתכנן את הקוד בעזרת החומר שנלמד ובדגש באופן שיאפשר הרחבה עתידית של הקוד. בפרט, יש לתת דגש על loose coupling ועל עקרונות ה SOLID. שימו לב, שבתרגילים הבאים תתבקשו לתחזק את הקוד הזה על ידי שינוי ותוספות שתתבקשו לממש. מימוש נכון, יביא לכך שיישום של הדרישות החדשות שיהיו יהיה יחסית פשוט למימוש.
- בפרט בדקו את הנק' הבאות:
- האם העובדה ששמות הפקודות השתנו הצריך מכם לגעת בקוד שאמור להיות "סגור לשינויים אבל פתוח להרחבה"?
- האם העובדה שנוספו פקודות חדשות הצריך מכם לגעת בקוד שאמור להיות "סגור לשינויים אבל פתוח להרחבה"?

- האם העובדה שפלט הפקודות השתנה הצריך מכם לגעת בקוד שאמור להיות "סגור לשינויים אבל פתוח להרחבה"?
- האם העובדה שהקלט/פלט מגיע מסוקטים במקום מהconsole הצריך מכם לגעת בקוד שאמור להיות "סגור לשינויים אבל פתוח להרחבה"?
- עליכם להוסיף לקובץ ה README התייחסות לנק' אלו ולהסביר לכל אחד מהם:
 - אם כן, כיצד תיקנתם את הקוד כך שלהבא שינויים כאלה לא יצריכו לגעת בקוד שהוא סגור לשינויים.
 - אם לא, מה היה הרעיון המימושי שלכם בתרגיל 1 שסייע לכם להרחיב את הקוד בלי לשנות אותו.
 (הקובץ README הוא כמובן באנגלית בלבד...)
- גם כאשר אתם עושים שינויים ותוספות בשביל התרגיל החדש, חישבו האם הם סגורים לשינויים ופתוחים להרחבה. ספציפית:
 - האם המימוש שלכם חייב לדעת כיצד multi-threading ממומש? כלומר, בתרגיל זה אתם פשוט יוצרים thread חדש לכל לקוח. **חובה לממש זאת כך בתרגיל זה שהשרת יוצר thread חדש לכל לקוח.** אבל, אם בעתיד (אהמ.. אהמ..) [משתעל] יבקשו מכם לשנות את זה, האם הקוד שלכם סגור לשינויים אבל פתוח להרחבות?
 - (כלומר, בתרגיל הבא אבקש מכם לשנות את זה ל thread pool, אבל במימוש נכון זה שינוי יחסית מינורי ולא מצריך שינוי בקוד קיים אלא רק להרחיב את הקוד הקיים)
- יש להקפיד את קוד נקי ומסודר, **מתועד**, כאשר יש הפרדה בכל המחלקות לקבצי `cpp` ו `h` והקפדה על מה צריך להיות כתוב ב `h` ומה צריך להיות כתוב ב `cpp`.
- קובץ ה README צריך להכיל הסברים מתאימים אודות האפליקציה, תמונות מתאימות, והסבר של איך מריצים ודוגמאת הרצה.
- הקפידו על חלוקה מסודרת של הפרוייקט. בתיקייה הראשית צריכה להיות תיקיית `src` אשר תכיל את הקוד שלכם ותיקיית `data` שתכיל את המידע שהקוד שלכם שומר/טוען. בנוסף, התיקייה הראשית צריכה להכיל קבצים שחייבים להיות ברמה הראשית, למשל, קובץ ה README.
- אני מזכיר, אין להעלות שום סיסמאות, מידע פרטי, רגיש או סודי, מפתחות, טוקנים, API Keys וכו' וכו' בשום מקום בפרויקט ובקורס. לא בgithub, לא בקוד, לא בREADME, לא ב WIKI לא בJIRA או בDOCKER ולא בשום מקום אחר. זיכרו שלרוב הדברים יש היסטוריה - והמידע מופיע גם שם, אפילו אם מחקתם אותו.
- חובה להגיש את התרגיל בזוגות או בשלושות. הגשה ביחידים (מכל סיבה שהיא), תגרור תקרה של ציון מקסימלי של 75 בכל תרגיל שיוגש ביחידים. לא ניתן יהיה לקבל ציון מעל 75 בתרגיל שהוגש ביחידים.
- חובה לעבוד בגיט (private repository) לאורך כל התרגיל. כל סטודנט עובד מהמחשב שלו ומהמשתמש הנפרד שלו. לא ניתן לעבוד ביחד מאותו מחשב או מאותו משתמש או ללא גיט. חובה לעבוד בשיטת feature branches בלבד ולתחזק branch ראשי שאליו ממרגי'ים את ה feature branches. חובה למרגי' קוד לbranch הראשי על ידי pull request בלבד, וחובה שמאשר ה pull request יהיה שונה ממי שביקש את ה pull request (במידה כמובן שאכן הבקשה אושרה - ייתכן ומי שבוחן את בקשת ה pull ימצא בעיות במימוש שיש לתקן טרם אישור

הבקשה). כל pull request חייב לעבור אישור של כל שאר חברי הקבוצה. כלומר, אם אתם זוג - אחד פותח pull request והשני עובר על הקוד ונותן הערות (אם ישנן) ורק אם הקוד תקין מאשר את הrequest. אם אתם שלשה - אחד פותח pull request והשניים האחרים צריכים לעבור על הקוד ונותנים הערות (אם ישנן) ורק אם הקוד תקין שניהם מאשרים את הrequest. דרך אגב, לgithub יש פיצ'ר מובנה שמאפשר לעשות את זה. אפשר להשתמש בו אבל לא חובה. (הוא זמין בחינם רק למי שעשה את החשבון כחשבון סטודנט, ואנחנו משתמשים רק בכלים חינמיים). אם עשיתם חשבון מתאים, אתם יכולים להשתמש בזה וזה פשוט לא יאפשר למי שפתח את הpull request לאשר לעצמו. אם בחרתם שלא להשתמש בפיצ'ר המובנה, באחריותכם להקפיד לבצע את הלוגיקה שהוגדרה (שאחד פותח PR והאחרים מאשרים). (למרג' == לעשות merge)

- המימוש של השרת (ולכן גם תכנון המשימות ב JIRA) חייב להיות לפי TDD. המימוש של הלקוח לא צריך להיות ב TDD, כלומר, לא צריך לכתוב טסטים עבור הלקוח. כמובן שכל שאר העבודה על הלקוח היא לפי כל אותם סטנדרטים של פיתוח השרת (משימות ב JIRA, פיצ'ר branches וכו').

- על ההגשה להכיל Dockerfile מתאים והוראות הרצה שלו כך שניתן יהיה להריץ בעזרת Docker את התרגיל, כלומר שרת בנפרד ולקוח בנפרד (ובנפרד, גם את ה unittests). ניתן להשתמש ב docker-compose למעוניינים.

- את התרגיל יש להגיש למודל לתיבת ההגשה על ידי הגשת קובץ טקסט בשם details.txt עם שמות ות.ז. של המגישים וקישור לגיט. שימו לב, חובה על הקובץ להיות בפורמט הבא:

Israel Israeli 123456789

Israela Israeli 012345678

Israeli Israeli 012345679

LINK TO GITHUB HERE

בלי רווחים נוספים, בלי שורות נוספות, ובשפה האנגלית בלבד. אי הגשה של קובץ ה details.txt הנ"ל או הגשתו באופן שונה ממה שהוגדר, תגרור הורדה של 20 נקודות בציון התרגיל.

- חובה להגיש את קובץ ה details למודל מיד עם תחילת העבודה. (משימה ב JIRA)

- אנא דאגו לכך שהגיט שלכם יהיה במצב private לכל אורך הסמסטר. יש להוסיף את המייל של המתרגלים (המייל המשותף שפירסמו לכם, לא המיילים הפרטיים שלהם) עם הרשאות גישה ל Github ול JIRA.

- אסור להשתמש בשום סיפרייה/מודול שאינו חלק מהספרייה הסטנדרטית שמגיעה עם C++ (למעט gtest) או שאינו חלק ממה שמגיע בילט-אין ב Python. אין לבקש בקשות בנוגע לתרגיל באופן מילולי. בקשות פרטניות רק דרך המייל של המתרגלים ושאלות כלליות על התרגיל אך ורק דרך הפורום. אין להשתמש בספריות שאינן טריוואליות. למשל, STL זה טריוויאלי וחלק מובנה ב C++, כך שאין בעיה להשתמש בו.

- שימו לב, אין לשנות את הrepo לאחר הדדליין. כל שינוי לאחר הדדליין ייחשב כאיחור.

- עבודה עצמית בלבד. הנושא ייבדק ויאכף. שימו לב, התרגיל הוא שלכם ואתם צריכים לממש אותו ולא כלים מבוססים Gen AI. שימוש בכלים מסוג זה באופן שכלים אלו מממשים עבורכם חלקים מהתרגיל שקול לעבודה שאינה עצמית ועצמאית. הנושא ייבדק ויאכף.

בהצלחה